

PROYECTO COMPLETO: AsadoPy

Actúa como un Staff Software Engineer, Product Designer, UX Designer y Tech Lead senior.

Tu tarea es desarrollar la totalidad de una aplicación web llamada **AsadoPy**, lista para producción, despliegue en Vercel y uso desde dispositivos móviles y escritorio.

No hagas preguntas. Toma todas las decisiones técnicas necesarias siguiendo las especificaciones de este documento.

Objetivo del proyecto

AsadoPy es una aplicación web que permite calcular automáticamente todos los insumos necesarios para realizar un asado paraguayo en función de los participantes.

La aplicación debe permitir:

- Gestionar una agenda de contactos.
 - Seleccionar participantes desde la agenda.
 - Agregar participantes manualmente.
 - Calcular automáticamente todos los insumos.
 - Mostrar una lista de compras clara y profesional.
 - Funcionar completamente sin backend.
 - Persistir datos localmente.
-

Stack tecnológico obligatorio

- Next.js 15+
 - React 19+
 - TypeScript
 - Tailwind CSS
 - shadcn/ui
 - Lucide Icons
 - Framer Motion
 - PWA instalable
 - LocalStorage o IndexedDB para persistencia
 - Deploy listo para Vercel
-

Requisitos generales

Mobile First

La aplicación debe diseñarse primero para celulares.

Debe funcionar perfectamente en:

- Android
 - iPhone
 - Tablets
 - Desktop
-

Diseño

Nivel comercial premium.

Incluir:

- Modo oscuro
 - Modo claro
 - Animaciones suaves
 - Tarjetas modernas
 - UX intuitiva
 - Componentes reutilizables
 - Diseño limpio y profesional
-

Módulo Agenda de Contactos

Crear un apartado llamado:

Contactos

Cada contacto debe contener:

- Nombre
- Sexo:
 - Hombre
 - Mujer
 - Niño
- Consume alcohol:
 - Sí
 - No
- Número de teléfono

Persistir contactos localmente.

Permitir:

- Crear contacto
 - Editar contacto
 - Eliminar contacto
 - Buscar contacto
-

Flujo principal

Paso 1

Seleccionar participantes desde la agenda.

Paso 2

Agregar participantes manualmente.

Paso 3

Permitir modificar para el evento actual:

- Nombre
- Sexo
- Consume alcohol

Sin modificar el contacto original.

Configuración de consumos

Antes de calcular, el usuario debe poder configurar valores personalizados.

Valores por defecto:

Carne

- Hombre: 350 g
- Mujer: 250 g
- Niño: 100 g

Chorizo

- Hombre: 150 g
- Mujer: 150 g
- Niño: 100 g

Cerveza

- Hombre: 3 litros
- Mujer: 2 litros
- Niño: 0 litros (prohibido tomar)

Todos estos valores deben poder modificarse.

Cálculos

Carne

Calcular según participantes.

Reglas:

- mínimo 1 kg
- aumentar de 100 en 100 gramos

Ejemplos:

1.03 kg → 1.1 kg

1.21 kg → 1.3 kg

Siempre redondear al siguiente múltiplo de 100 gramos.

Chorizo

Calcular según participantes.

Reglas:

- mínimo 0.5 kg
- aumentar de 100 en 100 gramos

Ejemplos:

0.63 kg → 0.7 kg

1.11 kg → 1.2 kg

Cerveza

Calcular únicamente participantes que consumen alcohol.

El usuario debe indicar:

- Tipo de envase
 - Lata
 - Botella
 - Botellita
 - Otro
- Capacidad del envase

La capacidad puede cargarse en:

- ml
- litros

Ejemplos:

- 269 ml
- 275 ml
- 1 litro
- 1.5 litros
- 600 ml
- 5 litros
- 330 ml

Calcular:

- Litros totales necesarios
- Cantidad de unidades necesarias

Fórmula:

$$\text{unidades} = \text{techo}(\text{litros_totales} / \text{capacidad_envase})$$

Mandioca

100 gramos por persona.

La mandioca se compra en bolsas de 1 kg.

Regla:

Si la parte decimal está entre:

0.1 y 0.4 → bajar

0.5 o más → subir

Ejemplos:

1.2 kg → 1 bolsa

1.4 kg → 1 bolsa

1.5 kg → 2 bolsas

1.8 kg → 2 bolsas

Ensalada

No calcular.

Mostrar:

"Llevar ensalada. Se recomienda que alguien del grupo la prepare ya que es difícil estimar cantidades."

Pan

150 gramos por persona.

Reglas:

- mínimo 0.5 kg
- aumentar de 100 en 100 gramos

Mostrar resultado en kg.

Carbón

Se vende en bolsas de:

- 3 kg
- 5 kg

Calcular:

$\text{carbon_necesario} = \text{carne_kg} + \text{chorizo_kg}$

Redondear siempre para arriba.

Generar automáticamente la mejor combinación de bolsas de:

- 3 kg
- 5 kg

Objetivo:

- Nunca faltar carbón.
- Minimizar el excedente.
- Si existen varias opciones válidas, priorizar la que tenga más carbón antes que menos.

Ejemplos:

Necesita 7 kg

Resultado:

1 bolsa de 5 kg

1 bolsa de 3 kg

Total: 8 kg

Pan de ajo

100 gramos por persona.

Se compra en paquetes de 400 gramos.

Regla:

Siempre redondear hacia arriba.

Ejemplos:

700 g → 2 paquetes

Sopa Paraguaya

100 gramos por persona.

Paquetes de 1 kg.

Regla:

0.1 a 0.4 → bajar

0.5 o más → subir

Hielo

Primero calcular:

litros_totales =
cerveza +
bebidas_sin_alcohol

Luego:

$\text{kg_hielo} = \text{litros_totales} / 2$

El hielo se compra en bolsas de 3 kg.

Resultado:

$\text{bolsas_hielo} = \text{techo}(\text{kg_hielo} / 3)$

Mostrar cantidad de bolsas.

Limón

Siempre:

6 unidades

Bebidas sin alcohol

Contar participantes que NO consumen alcohol.

Calcular:

500 ml por persona

Convertir a litros.

Redondear siempre hacia arriba.

Ejemplos:

3.5 litros → 4 litros

Mbeju

100 gramos por persona.

Paquetes de 1 kg.

Regla:

0.1 a 0.4 → bajar

0.5 o más → subir

Resultado final

Mostrar una pantalla de resumen profesional.

Ejemplo:

Lista de Compras

- Carne: 4.2 kg
- Chorizo: 1.8 kg
- Cerveza: 24 litros (34 latas de 710 ml)
- Mandioca: 2 bolsas
- Pan: 2.1 kg
- Carbón:
 - 2 bolsas de 5 kg
 - 1 bolsa de 3 kg
- Pan de ajo: 3 paquetes
- Sopa paraguaya: 2 paquetes
- Hielo: 3 bolsas
- Limón: 6 unidades
- Bebidas sin alcohol: 4 litros
- Mbeju: 2 paquetes

Funcionalidades adicionales

Implementar:

- Validaciones completas
 - Estados vacíos
 - Manejo de errores
 - Accesibilidad
 - SEO
 - Metadata
 - Manifest PWA
 - Service Worker
 - Íconos
 - Splash Screen
 - Instalación en Android y iPhone
-

Calidad de código

Exigir:

- Arquitectura escalable
 - Clean Architecture
 - TypeScript estricto
 - Componentes desacoplados
 - Hooks reutilizables
 - Testing de lógica de cálculo
 - Tipado completo
 - Código listo para mantenimiento
-

Entregables esperados

Generar:

1. Estructura completa del proyecto.
2. Todo el código fuente.
3. Componentes.
4. Hooks.
5. Utilidades de cálculo.
6. Tipos TypeScript.
7. Configuración PWA.
8. Configuración Vercel.
9. Tests.
10. Instrucciones de instalación.
11. Instrucciones de despliegue.

Desarrollar el proyecto completo sin dejar secciones como TODO, Placeholder o "implementar después".